

Question 1)

- I) The decision sequence of the problem consists of basins that provides minimum cost for transporting the water from the water source to field:

$$B_1, B_2, B_3, \dots, B_m$$
- II) The algorithm make decision at each step which of the basins between current basin and the water source can transport the water to current basin with the minimum cost. So, the alternatives at each step are the basins between the current basin and the water source.
- III) The algorithm calculates minimum cost to get water for each basin. Since the field can be considered as the last basin, the algorithm satisfies optimal solution for the problem. Otherwise, let B_o is the optimal basin to transport water to field with minimum cost. Let the algorithm doesn't satisfy the optimality and there is another basin B_k which is more basin than B_o . If the B_k is closer than B_o to the water source:

		B_k	K	B_o	L	Field
---	--	-------	-----	-------	-----	-------

So;

$$B_o + L - 200 > B_k + K + L - 200$$

$$B_o > B_k + K$$

Then B_o is not the optimal solution for basin "o" and it contradicts with the fact that the algorithm calculates optimal cost for each basin. If the B_o is closer than B_k to the water source:

		B_o	K	B_k	L	Field
---	--	-------	-----	-------	-----	-------

So;

$$B_o + (K + L) - 200 > B_k + L - 200$$

$$B_o + K > B_k$$

Then the algorithm should pick the B_k instead of B_o at the first as optimal basin and it causes a contradiction.

- IV) The algorithm calculates minimum cost of transporting the water to current basin at each step. The algorithm calculates the cost of each basin between the water source and the current basin by adding the cost of a basin itself and the cost to transport the water from the basin to the current basin. The cost of the current basin is equal to minimum of the calculated costs. The recurrence relation of the algorithm can be written as:

$$OPT[i] = \min_{0 \leq k \leq i} \{OPT[k] + |200 - (Basins[i] - Basins[k])|\}$$

, Where $OPT[0] = 0$

- V) The table is a $1 \times n$ array that contains optimal costs of having water for each basin. The table is filled by using the recurrence formula in part 4. OPT is the table in the formula and the i^{th} item of the array can be found by finding the minimum cost of

additions of these two costs: the optimal cost of item k and cost for transporting the water from basin k to i.

VI) # fill_OPT_table (basins):

Define the cost table as OPT

Set all OPT[i]'s to infinity

Set OPT[0] to 0

FOR i from 1 to n + 1:

FOR j from 0 to i:

current_cost = OPT[j] + | 200 – (basins[i] – basins[j]) |

IF current_cost < OPT[i]:

OPT[i] = current_cost

VII) Let assume that finding current_cost is an atomic operation and takes constant time. So, the complexity of the algorithm becomes:

For each value of i:

i's value	1	2	3	...	n	n+1
j loop count	1	2	3	...	n	n+1

$$\begin{aligned} \text{The total loop count of } j &= 1 + 2 + 3 + 4 + \dots + n + (n + 1) \\ &= \frac{(n + 1) * (n + 2)}{2} = O(n^2) \end{aligned}$$

VIII) Exact solution of the problem can be found by using the OPT table. The used basins for the optimal solution can be found by backtracking the OPT table.

#optimal_basin_set(basins, OPT):

Define the actual solution as opt_basins

FOR i from n+1 to 1:

FOR j from 0 to i:

current_cost = OPT[j] + | 200 – (basins[i] – basins[j]) |

IF current_cost == OPT[i] - OPT[j]:

Add basin j to head of opt_basins

Set i to j

Break

IX) Let assume that finding cost and adding the basins to actual solution can be done in a constant time. Since there are two nested loops with same boundaries as in the part 7, the complexity of the algorithm becomes $O(n^2)$.

Question 2)

- I) The DP algorithm makes decisions between the companies for each month to decide about staying the current company or switch the company. The algorithm tries to choose optimal (min cost) option for each month for both companies. Therefore, the decision sequence of the algorithm consists of the companies for each month for the both companies:

$$C_{11}, C_{12}, C_{13}, \dots, C_{1N}; C_{21}, C_{22}, C_{23}, \dots, C_{2N};$$

The number of decision steps is equal to number of months x 2 because, the billing period is a month and the algorithm makes two decisions for each month.

- II) The alternatives for each decision step are staying current company or swap the company. Therefore, there are four different decision option: Stay at company1, swap from company1 to company2, stay at company2 or swap from company2 to company1.
- III) The proposed DP algorithm always provides the optimal cost for the next month by calculating optimum costs of both company for the month. Optimal cost of company1 for the next month is equal to the minimum value of both costs: staying company1 for next month while already using the company1 on the current month and transfer data from company2 for the next month. In such a condition, the algorithm always provides the minimum cost of using company1 at any month. The fact is valid for the company2. So, the algorithm always provides the minimum(optimum) cost for both companies.
- IV) There are two recurrence relation for both companies separately because the optimal cost for a company at any month is related with both optimal plans of companies for the previous month. So, the recurrence relation is almost the same, but the optimal costs are stored different tables:

$$\begin{aligned} & \text{Optimal costs of company1} \\ & = OPTC_1[i] = \min\{OPTC_1[i-1], OPTC_2[i-1] + T\} + COSTC_1[i] \end{aligned}$$

$$\begin{aligned} & \text{Optimal costs of company2} \\ & = OPTC_2[i] = \min\{OPTC_2[i-1], OPTC_1[i-1] + T\} + COSTC_2[i] \end{aligned}$$

, Where both $OPTC_1[0]$ and $OPTC_2[0]$ are equal to zero

- V) There is two different 1D tables for the solution of the problem which hold optimal costs of the months for both companies. For example, k. value of table of company1 provides optimal total cost of using company1 at month k and it is same for the other table. The tables can be filled by using the recurrence formula: each value of the $OPTC_1$ (table of the company1) is the minimum value of its decision options: Staying at company1 while already using company1 at last month or pay the transfer cost and use company1 while using company2 at last month.

VI) # find_optimal_costs(costsc1, costsc2)

FOR i from 1 to N:

OPTC₁[i] = MIN(OPTC₁[i-1], (OPTC₂[i-1] + T)) + costc1[i]

OPTC₂[i] = MIN(OPTC₂[i-1], (OPTC₁[i-1] + T)) + costc2[i]

VII) Assume finding minimum of two value and addition is an atomic operation and can be done in a constant time $O(1)$. Since the memory access is done in constant time and the process loops N times, the complexity of the algorithm becomes:

$$O(N * 2) = O(N)$$

VIII) The exact solution of optimal plan can be found by backtracking. The backtracking algorithm compares optimum costs of Nth month of companies and picks the company that has the lower cost. After that, the algorithm finds out how the cost was handled. As mentioned before the cost can be handled by staying the same company at last month or there is a transfer from other company. The algorithm detects the company that is used on previous month by looking transfer cost and previous month optimal costs of companies.

find_optiamal_plan(OPTC₁, OPTC₂):

Define optimal_plan array

IF OPTC₁[N] < OPTC₂[N]:

optimal_plan[N] = current_company = C₁

ELSE:

optimal_plan[N] = current_company = C₂

FOR i from N to 2:

IF current_company == C₁:

IF OPTC₁[i] - T == OPTC₂[i-1]:

Optimal_plan[i-1] = current_company = C₂

ELSE:

Optimal_plan[i-1] = current_company = C₁

ELSE:

IF OPTC₂[i] - T == OPTC₁[i-1]:

Optimal_plan[i-1] = current_company = C₁

ELSE:

Optimal_plan[i-1] = current_company = C₂

IX) The backtracking algorithm loops N-2 times and do 2 comparisons and one memory access at every loop. Let assume the operations can be done in a constant time $O(1)$. The complexity of the algorithm becomes:

$$\theta((N - 2) * 3) = \theta(N)$$

Question 3)

- I) The DP algorithm makes decisions about the any substring between i^{th} and j^{th} characters of the text is a palindrome or not. In other words, the algorithm decides the start and end index of the palindromic substrings. If there is a palindrome between $i+1 - j-1$ characters with length k and i . character is same with the j . character, then the algorithm decides the boundaries of the palindrome can be extended to i and j . On the other hand, if the i . and j . characters are different from each other, the algorithm stops to extend the substring as a palindrome. So, the decision sequence of the algorithm becomes a sequence of palindrome lengths:

$$L_{01}, L_{12}, L_{23}, \dots, L_{1n-1}, L_{0n}$$

- II) Since the algorithm decides the length of the palindrome between i and j characters, the alternatives are the lengths between 0 and $|j-i|+1$.
- III) The algorithm provides to give length of palindrome between i and j by the length of longest palindrome between $i+1$ and $j-1$ and the i . and j . characters. Let assume there is a palindromic substring P_{other} between $i-j$ which is longer than the palindromic substring P_{offered} that is proposed by the algorithm. If $i-1$ and $j+1$ characters are the same then, Length of P_{offered} extends and becomes $\text{Len}(P_{\text{offered}}) + 2$ which is equal to $(j+1) - (i-1)$. On the other hand, P_{other} also should be extended by the new same characters and the length of the new palindromic substring becomes longer than the length of substring between $i-1$ and $j+1$ which is contradict with the fact that length of a string can not be more than its characters count.
- IV) The recurrence relation satisfies that the substring between i . and j . characters is a palindromic substring or not:

$$L_{i,j} = L_{i+1,j-1} \cap (\text{Text}[i] == \text{Text}[j]) \\ , \text{where } i \leq j \text{ and } L_{ii} = \text{True}$$

- V) The table for the DP algorithm is an $n \times n$ table which consists of true and false. Each cell L_{ij} represents whether there is a palindromic substring between characters i and j . First the cells on the diagonal fills because each single character is a palindromic substring. Then fill the cells which are on the diagonal by applying the recurrence formula. The rows in the table represents start character and the columns are the end characters of sub strings.

$i \backslash j$	0	1	..	n-1	n
0	True				
1		True			
...			True		
n-1				True	
n					True

VI) # fill_table(text)
 Define the Table
 FOR i from 0 to n:
 Table[i, i] = True
 FOR i from 0 to n - 1:
 IF text[i] == text[i+1]:
 Table[i, i+1] = True
 FOR i from 3 to n:
 FOR j from 0 to n-i+1:
 k = i + j -1
 IF Table[j+1, k-1] == True and text[j] == text[k]:
 Table[j, k] = True

VII) The first two loop depends to n so the complexity of the first two loop is $\Theta(n)$. The third loop is a nested loop which consists of two loops. The first loop iterates n times and the loop count of inner loop is depends on the value of i:

Value of i	3	4	5	...	n-1	N
Loops count of inner loop	n-2	n-3	n-4	...	2	1

$$\begin{aligned}
 \text{Total loop count} &= n - 2 + n - 3 + \dots + 2 + 1 = \frac{(n - 2) * (n - 1)}{2} \\
 &= \frac{n^2 - 3n + 2}{2} = \Theta(n^2)
 \end{aligned}$$

Since the n^2 is more complex than n, the complexity of the whole algorithm is $\Theta(n^2)$.

VIII) Since the Table[i,j] holds the information that the substring of text between i and j. characters is palindromic or not, The longest palindromic substring can be found by traversing the rows of the table from right to left. Since there is a true cell at table[i,j] that means the longest palindromic substring that starts with i. character ends with the j. character.

```

# find_longest_palindromic (Table)
max_len = 0
start_index=0
FOR i from 0 to n:
    FOR j from n-1 to i:
        IF Table[i, j]==True:
            IF j - i > max_len:
                max_len = j - i
                start_index = i

```

- IX) There is a nested loop that influence the complexity of the algorithm. The first loop loops n time and the loops count of second loop changes by the value of n:

Value of i	0	1	2	...	n-1
Loops count of inner loop	n-1	n-2	n-3	...	1

$$\begin{aligned}
 \text{Total loop count} &= n - 1 + n - 2 + \dots + 2 + 1 \\
 &= \frac{(n - 1) * (n)}{2} = \frac{n^2 - n}{2} = \theta(n^2)
 \end{aligned}$$